

랭체인 활용

Data connection

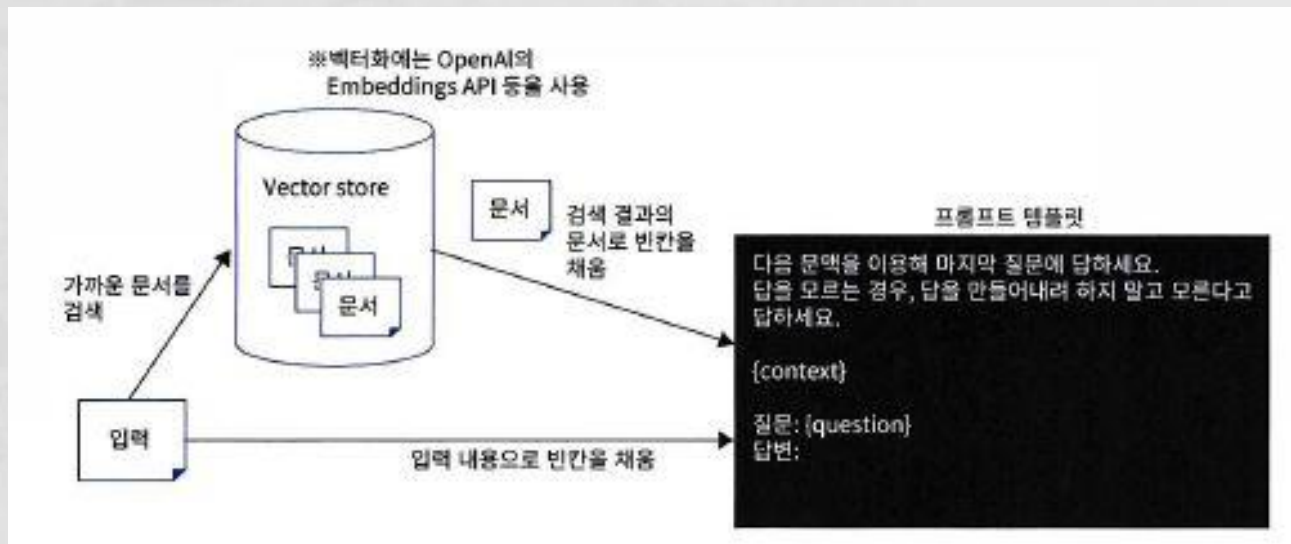
Data connection

- Data connection은 LLM과 외부의 데이터를 연결하기 위한 기능
- 입력과 관련이 있을 것 같은 문서를 검색하여 컨텍스트에 포함시키는 방법
- 문서를 OpenAI Embeddings API 등으로 벡터화해 입력과 벡터가 가까운 문서를 검색하여 컨텍스트에 포함시키는 기법 RAG(Retrieval Augmented Generation)
- 문서는 미리 준비된 데이터베이스에서 검색할 수도 있고, 구글과 같은 검색엔진으로 웹상에서 검색 할 수도 있음

Data connection

Data connection

- 랭체인의 Data connection에서는 특히 벡터 스토어(Vector store)를 사용하여 문서를 벡터화하여 저장해 두었다가
- 입력된 텍스트와 벡터에 가까운 문서를 검색하여 컨텍스트에 포함시켜 사용하는 방법을 제공



Data connection

Data connection

- 텍스트 벡터화란 텍스트를 숫자 배열로 변환하는 것을 말함
- 텍스트 벡터화에는 다양한 방법
- 일반적으로 등장하는 키워드나 의미가 가까운 텍스트가 벡터로도 거리가 가까워지도록 변환
- 텍스트 벡터화 자체는 최근에 등장한 기술이 아니라 자연어 처리 분야에서 오래전부터 자주 사용

Data connection

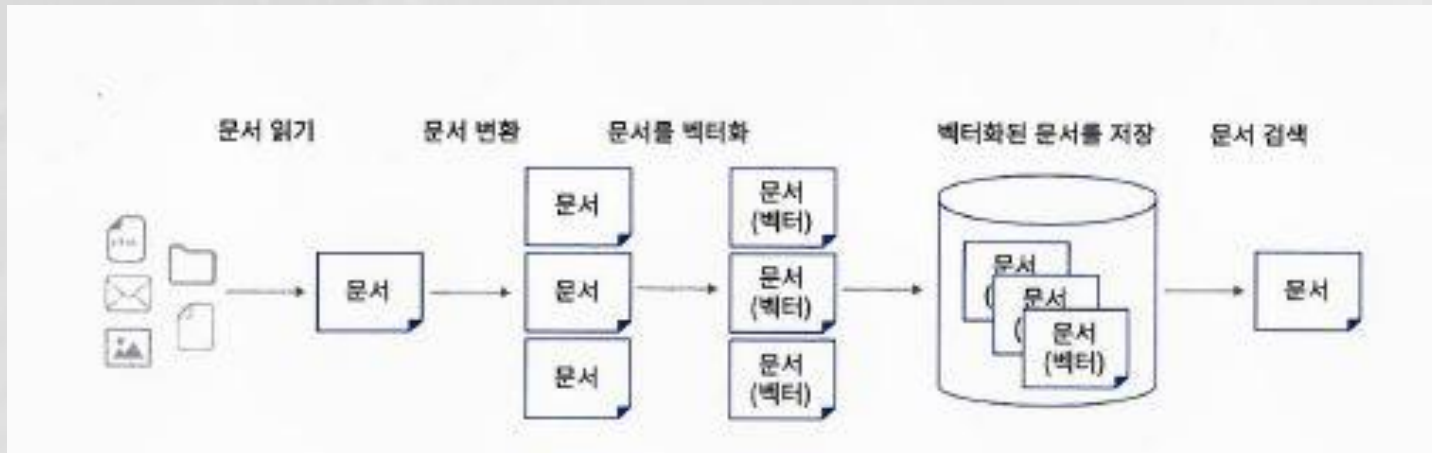
Data connection 개요

- RAG에 사용할 수 있는 랭체인 모듈은 “Data connection”
- Data connection에는 다음과 같은 5가지 기능이 있음
 - Document loaders : 데이터 소스로부터 문서를 불러온다.
 - Document transformers : 문서에 어떤 종류의 변환을 가한다.
 - Text embedding models : 문서를 벡터화한다.
 - Vector stores : 벡터화된 문서를 저장할 수 있는 저장소
 - Retrievers : 입력한 텍스트와 관련된 문서를 검색한다

Data connection

Data connection 개요

- 정보원(출처)이 되는 데이터부터 Retriever를 통한 검색까지 연결



!pip install pydantic

!pip install langchain openai langchain-openai langchain-community

!pip install langchain-classic

Data connection

Document loaders

- 랭체인을 문서 불러와야 함
- 데이터 로딩에 사용하는 것이 "Document loaders"
- GitPython이라는 패키지가 필요하므로 설치

!pip install GitPython

Data connection

Document loaders

- GitLoader를 사용해 랭체인 저장소에서 .md라는 확장자를 가진 파일을

로드

```
from langchain_classic.document_loaders import GitLoader

def file_filter(file_path):
    return file_path.endswith(".md")

loader = GitLoader(
    clone_url="https://github.com/langchain-ai/langchain",
    repo_path="./langchain",
    branch="master",
    file_filter=file_filter,
)

raw_docs = loader.load()
print(len(raw_docs))
```

36

Data connection

Document loaders

◦ 랭체인 DocumentLoader

DocumentLoader	개요
UnstructuredFileLoader	텍스트 파일, 파워포인트, HTML, PDF, 이미지 등 파일 불러오기
DirectoryLoader	UnstructuredFileLoader 등으로 디렉터리 내 모든 파일 불러오기 (UnstructuredFileLoader)
SitemapLoader	사이트맵에 따라 웹사이트의 각 페이지 불러오기
S3DirectoryLoader	Amazon S3의 버킷을 지정하여 오브젝트 불러오기
GitLoader	Git 저장소에서 파일 불러오기
BigQueryLoader	Google BigQuery에 SQL을 발행하여 행 단위로 문서로 불러오기
GoogleDriveLoader	구글 드라이브에서 파일 불러오기
ConfluenceLoader	Confluence 페이지 불러오기
NotionDirectoryLoader	Notion에서 export한 파일 불러오기
SlackDirectoryLoader	Slack에서 export한 파일 불러오기
HuggingFaceDatasetLoader	Hugging Face Hub에서 데이터 세트 불러오기

Data connection

Document transformers

- DocumentLoader에서 불러온 데이터를 '문서'
- 문서에 어떤 변환을 가하는 것이 'Document transformers'
- 문서를 일정 길이의 청크(분할된 텍스트 하나하나)로 분할하고 싶을 때
- 문서를 적절한 크기의 청크로 분할하면 LLM에 입력하는 토큰 수를 줄이고 더 정확한 답변을 얻을 수 있음

Data connection

Document transformers

- 랭체인을 CharacterTextSplitter 클래스를 사용해 문서를 청크로 분할

```
from langchain_classic.text_splitter import CharacterTextSplitter

text_splitter = CharacterTextSplitter(chunk_size=1000, chunk_overlap=0)

docs = text_splitter.split_documents(raw_docs)
print(len(docs))
```

- tiktoken에서 측정한 토큰 수로 분할하거나 파이썬 등의 소스 코드를 가
능한 한 클래스나 함수처럼 묶어서 분할하는 기능도 제공

Data connection

Document transformers

- 문서를 청크로 분할하는 것 외에도 몇 가지 변환 프로세스가 지원

DocumentTransformer	개요
Html2TextTransformer	HTML을 일반 텍스트로 변환하기
EmbeddingsRedundantFilter	유사한 문서 제외하기
OpenAIMetadataTagger	메타데이터 추출하기
DocumentTransformer	개요
DoctranTextTranslator	문서 번역하기
DoctranQATransformer	문서에서 Q&A를 생성하여 사용자의 질문과 쉽게 연관시키기

Data connection

Text embedding models

- 문서 변환 처리를 마쳤다면 텍스트를 벡터화할 차례
- OpenAI의 Embeddings API를 사용하여 text-embedding-ada-002라는 모델로 텍스트를 벡터화

```
embedding = client.embeddings.create(  
    model="text-embedding-ada-002",  
    input="Langchain이란 무엇인가?"  
)  
vector = embedding.data[0].embedding
```
- LangChain에는 OpenAI의 Embeddings API를 래핑한 OpenAIEmbeddings라는 클래스
- OpenAIEmbeddings와 마찬가지로 텍스트 벡터화에 사용할 수 있는 것이 'Text embedding models'

Data connection

Text embedding models

- OpenAIEmbeddings의 인스턴스를 생성
- 문서의 벡터화 처리는 Vector store 클래스에 데이터를 저장할 때 내부적으로 수행

```
from langchain_openai import OpenAIEmbeddings  
  
embeddings = OpenAIEmbeddings()
```

```
!pip install tiktoken
```


Data connection

Text embedding models

○ OpenAIEmbeddings를 사용해 텍스트를 벡터화

```
query = "AWS의 S3에서 데이터를 읽기 위한 DocumentLoader가 있나요?"
```

```
vector = embeddings.embed_query(query)
print(len(vector))
print(vector[:100])
```

1536

```
[-0.01556143257766962, -0.012867593206465244, 0.021737877279520035, -0.01614966429769993, -0.024117546156048775, 0.045186977833509445,
-0.019705800339579582, -0.0077606625854969025, -0.0040641543455421925, 0.022138945758342743, 0.01052134670317173, 0.01549458783119917,
0.0010402697371318936, -0.018796712160110474, -0.007393017411231995, -0.002800790360197425, 0.0177271980792284, -0.00953204557299614, -
0.009244613349437714, -0.03665760159492493, 0.012967860326170921, -0.015628276392817497, -0.005621634423732758, 0.0017429740400984883,
-0.0026554034557193518, 0.009017341770231724, 0.007687133736908436, -0.04016026109457016, 0.007219220977276564, 0.0026470478624105453,
0.0526735782623291, -0.013041389174759388, -0.0068415491841733456, -0.022727178409695625, -0.010026696138083935, -0.014705820940434933,
0.00838231761008501, -0.0006972731789574027, -0.006173102650791407, -0.02152397483587265, 0.002012023702263832, -0.006493956781923771,
-0.0006116284639574587, -0.04347575455904007, 0.002316166879609227, 0.02005339227616787, 0.002648718887940049, -0.04251319169998169, -
0.007981250062584877, 0.01530742272734642, 0.003452525706961751, 0.02478599362075329, -0.015588169917464256, -0.0372190959751606, -0.02
6858177036046982, 0.012159040197730064, -0.016644315794110298, 0.0018749921582639217, -0.007339541334658861, -0.027834108099341393, 0.0
07573497947305441, 0.033235155045986176, -0.01822184957563877, -0.005220566410571337, 0.002093908376991749, -0.007693818304687738, -0.0
10327496565878391, 0.0034658946096897125, 0.009565467946231365, -0.011397011578083038, 0.005434469319880009, 0.03018704056739807, 0.004
468564409762621, -0.02255338244140148, 0.03572177514433861, -0.005076850298792124, -0.009812792763113976, -0.01508015114814043, -0.0318
18047165870667, -0.004197843372821808, 0.012312782928347588, -0.01720581017434597, 0.00178809417411685, -0.005912408232688904, 0.003943
833988159895, 0.020441090688109398, 0.015213840641081333, 0.02868971973657608, -0.008455847389996052, -0.008495953865349293, -0.0096389
96794819832, -0.0004908903501927853, 0.01054139994084835, 0.015240577980875969, 0.020815420895814896, 0.02199188619852066, -0.013903684
914112091, 0.027138924226164818, -0.01663094572722912, -0.040427640080451965]
```

Data connection

Vector stores

- 저장할 Vector store를 준비하여 문서를 벡터화하여 저장
- 크로마(Chroma)라는 로컬에서 사용 가능한 벡터 스토어를 사용
- 크로마를 사용하기 위해 필요한 패키지를 설치

!pip install chromadb

Data connection

(참고) 크로마 DB



- chroma는 CRUD를 위해 다음과 같이 제공
- add : 새로운 문서(document)와 임베딩을 컬렉션에 추가
- update : 이미 존재하는 ID의 document / metadata를 수정
- upsert : ID가 있으면 → update, ID가 없으면 → add
- query : 입력 쿼리와 의미적으로 가장 유사한 문서 검색
- get : 특정 ID의 데이터를 그대로 조회
- delete : 특정 ID 또는 조건에 맞는 데이터 삭제

```
import chromadb
from chromadb.config import Settings

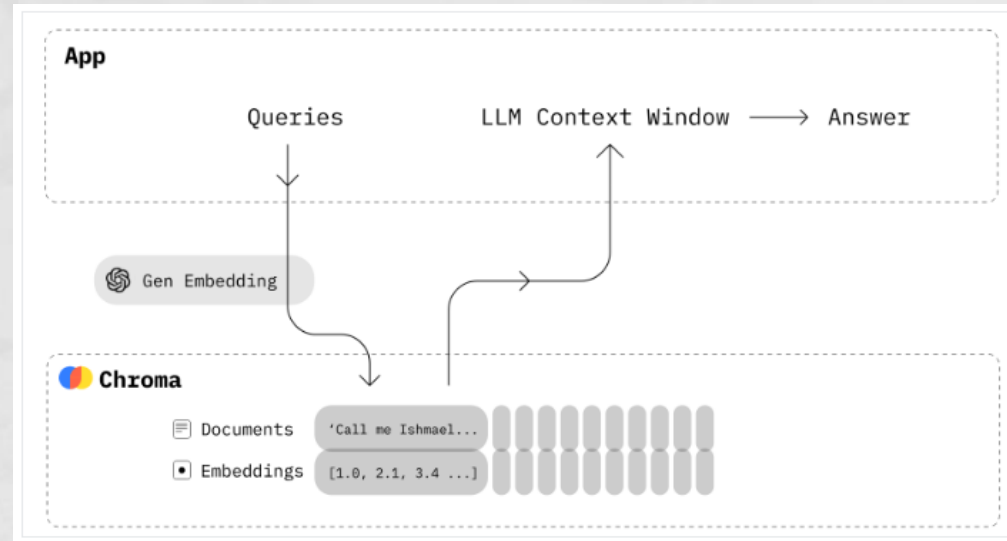
# 클라이언트 초기화 및 설정
client = chromadb.Client(Settings(
    chroma_db_impl="duckdb+parquet", # DuckDB 사용
    persist_directory="db/"          # 데이터 저장 디렉토리 지정
))
```

```
posts = client.create_collection(
    name="posts"
)
```

Data connection

(참고) 크로마 DB

```
collection = chroma_client.get_collection("new_collection")  
results = collection.query(  
    query_texts=["This is a query document about florida"],  
    # Chroma will embed this for you  
    query_embeddings=[get_sentence_embedding("This is a query I'm JeongT")], # Or you can embed  
    it yourself  
    n_results=1 # how many results to return  
)
```



Data connection

Vector stores

- 청크로 분할한 문서와 Text embedding model을 기반으로 벡터 데이터 베이스를 초기화

```
from langchain_classic.vectorstores import Chroma  
  
db = Chroma.from_documents(docs, embeddings)
```

- 문서를 벡터화하여 벡터 스토어에 저장
- 랭체인에서는 크로마 외에도 Faiss, Elasticsearch, Redis 등 벡터 데이터 베이스로 사용할 수 있는 많은 통합이 제공

Data connection

Retrievers

- 벡터 데이터베이스에서는 사용자의 입력과 관련된 문서를 가져오는 작업을 수행
- 랭체인에서 텍스트와 관련된 문서를 가져오는 인터페이스를 'Retriever'

```
retriever = db.as_retriever()
```

```
query = "AWS S3에서 데이터를 불러올 수 있는 DocumentLoader가 있나요?"
```

```
context_docs = retriever.invoke(query)  
print(f"len = {len(context_docs)}")
```

```
first_doc = context_docs[0]  
print(f"metadata = {first_doc.metadata}")  
print(first_doc.page_content)
```

Data connection

Retrievers

- 4개의 문서가 발견되었는데, 그중 첫 번째 문서는

'docs/docs/integrations/platforms/aws.mdx'로 AWS의 S3를 대상으로 하는 DocumentLoader에 대한 내용

```
len = 4
metadata = {'file_type': '.md', 'file_name': 'README.md', 'source': '.devcontainer\\README.md', 'file_path': '.devcontainer\\README.m
d'}
```

Then you will have a local cloned repo where you can contribute and then create pull requests.

If you already have VS Code and Docker installed, you can use the button above to get started. This will use VSCode to automatically install the Dev Containers extension if needed, clone the source code into a container volume, and spin up a dev container for use.

Alternatively you can also follow these steps to open this repo in a container using the VS Code Dev Containers extension:

1. If this is your first time using a development container, please ensure your system meets the pre-reqs (i.e. have Docker installed) in the [getting started steps](<https://aka.ms/vscode-remote/containers/getting-started>).
2. Open a locally cloned copy of the code:

Data connection

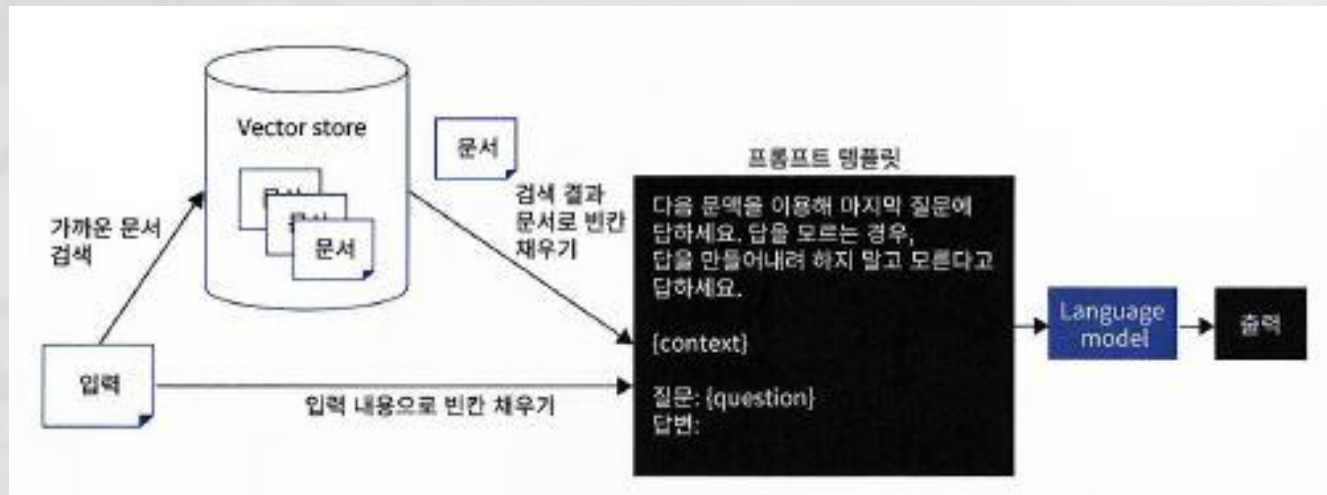
Retrievers

- Retriever 내부에서는 주어진 텍스트(query)를 벡터화하여 Vector 데이터베이스에 저장된 문서 중 벡터 거리가 가까운문서를 찾고 있음
- 문서를 벡터화하여 저장해두고, 사용자의 입력에 가까운 문서를 검색 (Retrieve)하는 과정
- 챗봇과 같은 애플리케이션에서는 입력과 관련된 문서를 검색(Retrieve)하는 것 외에도 검색한 내용을 PromptTemplate에 context로 삽입하여 LLM에게 질문하고 답변(QA)을 요청하는 경우

Data connection

Retrieval QA

- 일련의 처리를 위해 RetrievalQA라는 이름의 Chain이 제공



- 'stuff', 'map_reduce', 'map_rerank', 'refine'은 처리할 수 있는 문서의 길이, LLM 호출 횟수, 처리의 병렬화 여부 등의 차이가 있어 상황에 따라 구분하여 사용

Data connection

Retrieval QA

RetrievalQA를 사용

```
from langchain_classic.chains import RetrievalQA
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model_name="gpt-4.1-mini", temperature=0)
qa_chain = RetrievalQA.from_chain_type(llm=llm, chain_type="stuff", retriever=retriever)

qa_chain.invoke(query)
```

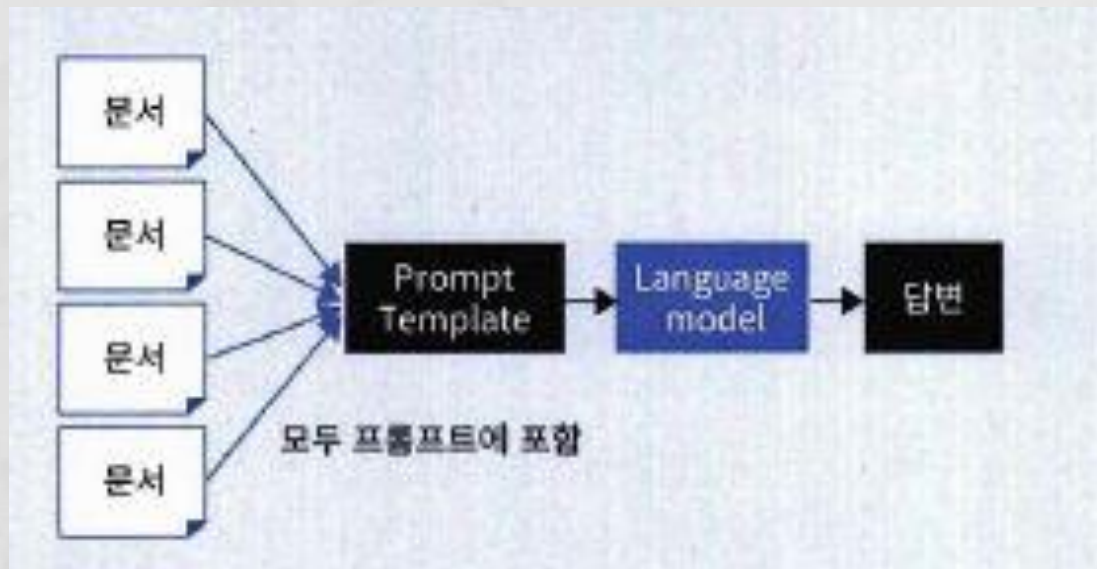
이 코드를 실행하면 LLM의 답변은 Retriever로 검색한 텍스트를 바탕으로 답변

```
{'query': 'AWS S3에서 데이터를 불러올 수 있는 DocumentLoader가 있나요?',
 'result': '네, LangChain에는 AWS S3에서 데이터를 불러올 수 있는 DocumentLoader가 있습니다.\n\n`langchain.document_loaders.s3` 모듈 내에 `S3FileLoader` 클래스를 사용하면 S3 버킷에 저장된 파일을 로드할 수 있습니다.\n\n예를 들어, 다음과 같이 사용할 수 있습니다:\n\n```\npython\nfrom langchain.document_loaders import S3FileLoader\n\n# S3 버킷 내 파일 경로 지정 (예: `my-bucket/path/to/file.txt`)\ns3_path = "s3://my-bucket/path/to/file.txt"\n\nloader = S3FileLoader(s3_path)\ndocuments = loader.load()\n```\n\n이때 AWS 자격 증명은 환경 변수나 AWS CLI 설정을 통해 미리 설정되어 있어야 합니다.\n\n더 자세한 내용은 [통합 문서](https://docs.langchain.com/oss/python/integrations/providers)에서 확인할 수 있습니다.'}
```

Data connection

RetrievalQA의 chain_type

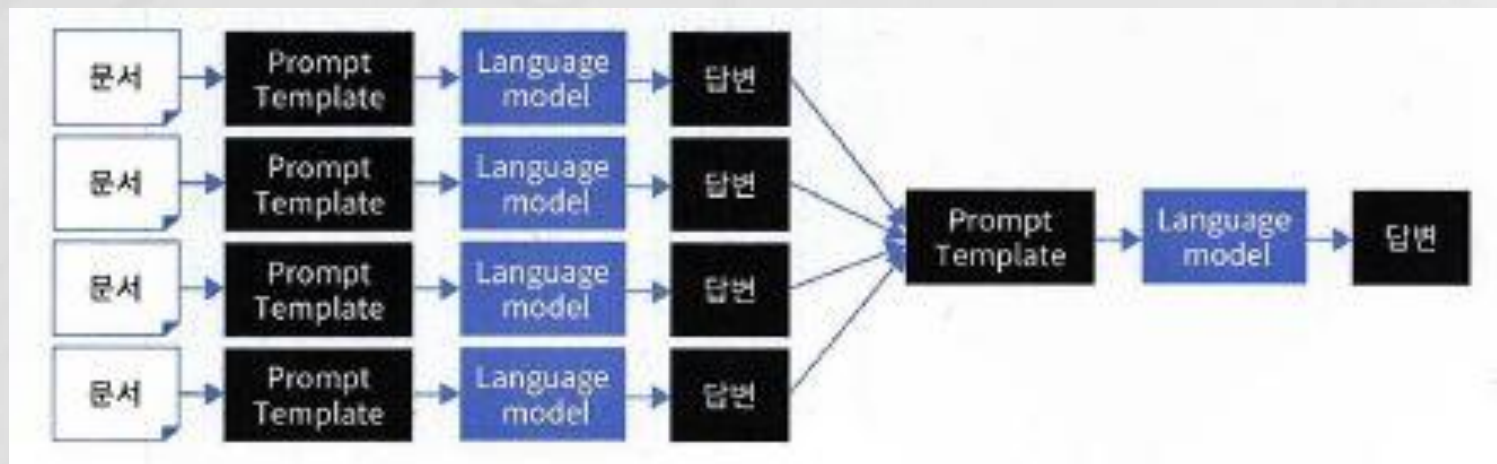
- chain_type으로 'stuff'를 지정하여 입력과 관련된 여러 문서를 동시에 프롬프트에 포함시켜 답변을 얻을 수 있게 되었음



Data connection

RetrievalQA의 chain_type

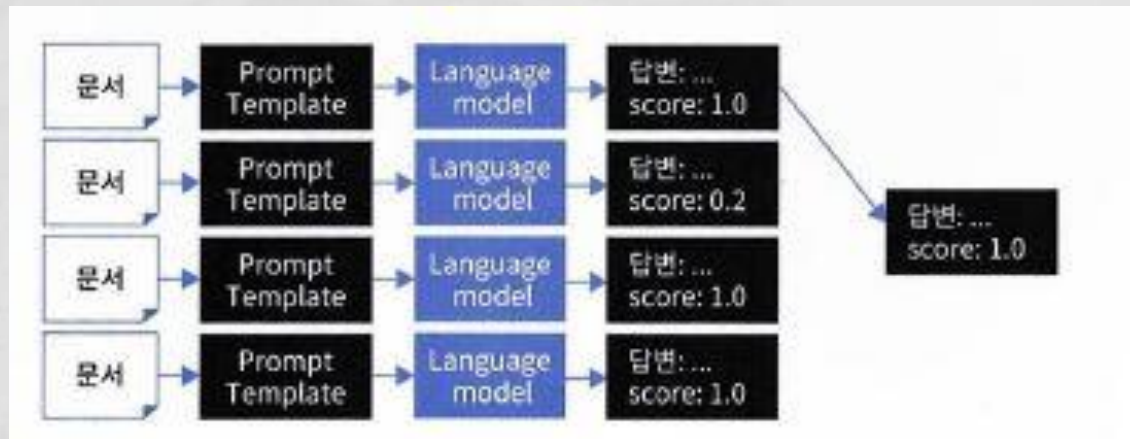
- chain_type으로 map_reduce를 선택하면 각 문서에 대한 답변을 얻고 (map), 이후 최종 답변을 얻는 (reduce) 흐름



Data connection

RetrievalQA의 chain_type

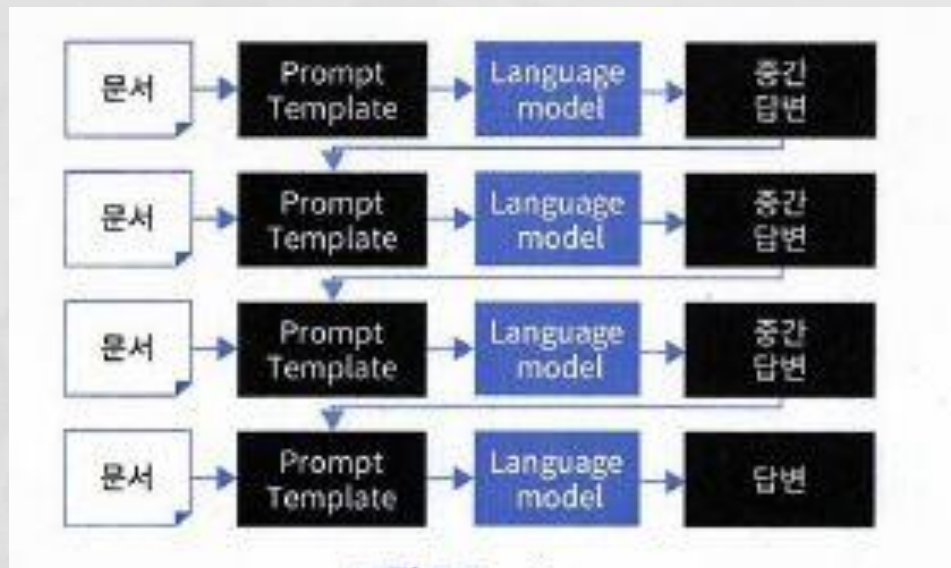
- chain_type으로 map_rerank를 선택하면 map_reduce와 마찬가지로 각 문서에 대한 답변을 얻는 동시에 LLM이 답변에 점수를 매김
- 그 점수가 가장 높은 답변이 최종 답변으로 채택



Data connection

RetrievalQA의 chain_type

- chain_type에 refine를 지정한 경우, LLM에 문서를 하나씩 주면서 점진적으로 답변을 만들게 함
- 즉, 점진 적으로 답변을 다듬어 나가는 흐름이 된다.



Agents

Agents 개요

- RetrievalQA(Chain)에서는 Vector store를 검색하고 검색 결과를 바탕으로 LLM이 응답
- 랭체인에는 LLM의 응답을 기반으로 API 호출하는 OpenAPIEndpointChain 과 SQL을 실행하는 SQLDatabaseChain과 같은 Chain도 있음
- 이러한 Chains는 고정된 처리 흐름을 구현
- 어떤 처리를 할 것인지 LLM이 선택해서 움직여 주기를 원하는 경우

Agents

Agents 개요

- 사용자가 입력한 내용을 바탕으로 필요에 따라 사내 문서를 Vector store에서 검색하여 답변하거나 웹상의 정보를 검색하여 답변
- 작동을 실현할 수 있는 것이 바로 랭체인 'Agents'
- 랭체인 Agents를 사용하면 필요에 따라 다양한 도구를 사용하면서 LLM을 작동시킬 수 있음
- 도구로는 Vector store를 통해 특정 분야의 데이터를 검색하여 사용하게 할 수도 있고, 구글과 같은 검색 엔진의 API를 사용하게 할 수도 있음

Agents

Agents 사용 예시

- 랭체인에는 다양한 종류의 Agents가 구현되어 있는데, 여기서는 ReAct 라는 종류의 Agents를 사용하는 예제를 구현

```
import os
from dotenv import load_dotenv
load_dotenv()
OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")
```

```
!pip install langchain openai langchain-openai langchain-experimental langchainhub
```

Agents

Agents 사용 예시

- 랭체인에는 다양한 종류의 Agents가 구현되어 있는데, 여기서는 ReAct 라는 종류의 Agents를 사용하는 예제를 구현

```
import langchain
```

```
langchain.verbose = True
```

```
from langchain_classic import hub
```

```
from langchain_classic.agents import AgentExecutor, create_react_agent, load_tools
```

```
from langchain_openai import ChatOpenAI
```

```
llm = ChatOpenAI(model_name="gpt-4.1-mini", temperature=0)
```

```
tools = load_tools(["terminal"], allow_dangerous_tools=True)
```

```
prompt = hub.pull("hwchase17/react")
```

```
agent = create_react_agent(llm, tools, prompt)
```

```
agent_executor = AgentExecutor(agent=agent, tools=tools, verbose=True)
```

```
result = agent_executor.invoke({"input": "sample_data 디렉터리에 있는 파일 목록을 알려줘, 터미널을 사용할 때 인코딩도 알아서 맞춰서 해줘"})
```

Agents

Agents 사용 예시

- 랭체인에는 다양한 종류의 Agents가 구현되어 있는데, 여기서는 ReAct 라는 종류의 Agents를 사용하는 예제를 구현

```
import langchain
```

```
langchain.verbose = True
```

```
from langchain_classic import hub
```

```
from langchain_classic.agents import AgentExecutor, create_react_agent, load_tools
```

```
from langchain_openai import ChatOpenAI
```

```
llm = ChatOpenAI(model_name="gpt-4.1-mini", temperature=0)
```

```
tools = load_tools(["terminal"], allow_dangerous_tools=True)
```

```
prompt = hub.pull("hwchase17/react")
```

```
agent = create_react_agent(llm, tools, prompt)
```

```
agent_executor = AgentExecutor(agent=agent, tools=tools, verbose=True)
```

```
result = agent_executor.invoke({"input": "sample_data 디렉터리에 있는 파일 목록을 알려줘, 터미널을 사용할 때 인코딩도 알아서 맞춰서 해줘"})
```

Agents

Agents 사용 예시

- load_tools라는 함수로 'terminal'이라는 도구를 준비
- 'terminal'은 셸에서 명령을 실행할 수 있는 도구
- create_react_agent라는 함수를 사용해 ReAct Agent를 초기화하고, sample_data 디렉터리에 있는 파일 목록을 알려달라고 질문
- Agents를 사용한 실행 결과에서는 실제로 지정한 디렉터리에 있는 파일

목록이 표시

```
Thought: sample_data 디렉터리에 있는 파일 목록을 확인했다. 파일들은 dog1.webp,
g, dog8.jpg, dog9.jpg 이다.
Final Answer: sample_data 디렉터리에 있는 파일 목록은 다음과 같습니다:
- dog1.webp
- dog2.jpg
- dog3.jpg
- dog4.jpg
- dog5.png
- dog6.jpg
- dog7.jpg
- dog8.jpg
- dog9.jpg

> Finished chain.
```

Agents

Agents의 작동 원리와 ReAct 개념

- ReAct Agent는 'ReAct:Synergizing Reasoning and Acting in Language Models'라는 논문에 소개한 메커니즘으로 작동
- ReAct : 언어 모델에서 추론과 행동의 시너지를 발휘하는 언어 모델
- 'Action:'을 써서 액션을 지정

```
I can use the terminal to list the files in the sample_data directory.  
Action: terminal  
Action Input: ls sample_data
```
- 'Action Input:'을 써서 액션 입력을 지정
 - 'Action'으로 'terminal'이 지정돼 있고, 'Action Input'으로 dir sample_data'가 지정
- 랭체인인 Agent는 이 응답 문자열에서 Action과 Action Input의 내용을 정규식으로 추출

Agents

Agents의 작동 원리와 ReAct 개념

- Agent는 원래의 프롬프트에 LLM의 응답과 액션 실행 결과를 추가한 프롬프트를 생성

Thought: sample_data 디렉터리에 있는 파일 목록을 확인하기 위해 터미널 명령어를 사용하겠다. 인코딩 문제를 방지하기 위해 기본적으로 UTF-8 인코딩을 가정하고 명령어를 실행하겠다.

Action: terminal

Action Input: dir sample_dataExecuting command:

dir sample_data

Thought: dir 명령어를 실행했으나 결과가 표시되지 않았다. 인코딩 문제나 출력 버퍼 문제일 수 있으니, PowerShell 명령어로 다시 시도해보겠다.

Action: terminal

Action Input: powershell -Command "Get-ChildItem -Path sample_data | Select-Object Name"Executing command:

powershell -Command "Get-ChildItem -Path sample_data | Select-Object Name

Name

dog1.webp

dog2.jpg

dog3.jpg

dog4.jpg

dog5.png

Answer the following questions as best you can. You have access to the following tools:

terminal: Run shell commands on this Linux machine.

Use the following format:

Question: the input question you must answer

Thought: you should always think about what to do

Action: the action to take, should be one of [terminal]

Action Input: the input to the action\nObservation: the result of the action

... (this Thought/Action/Action Input/Observation can repeat N times)

Thought: I now know the final answer

Final Answer: the final answer to the original input question

Begin!

Question: sample_data 디렉터리에 있는 파일 목록을 알려줘

Thought:I should use the terminal to list the files in the sample_data directory.

Action: terminal

Action Input: ls sample_data

Observation: anscombe.json

california_housing_test.csv

california_housing_train.csv

mnist_test.csv

mnist_train_small.csv

README.md

Thought:"

Agents

Agents의 작동 원리와 ReAct 개념

- Question
- ↓
- Thought (어떻게 풀지 생각)
- ↓
- Action (Tool 선택)
- ↓
- Action Input (Tool에 넘길 입력)
- ↓
- Observation (Tool 실행 결과)
- ↓
- Thought
- ↓
- Final Answer

질문 : "서울의 오늘 날씨는?"

Thought: 오늘 서울 날씨를 알아야 한다. 검색이 필요하다.

Action: Search

Action Input: "서울 오늘 날씨"

Observation: 오늘 서울은 맑고 최고 25도이다.

Thought: 검색 결과를 기반으로 답할 수 있다.

Final Answer: 오늘 서울은 맑고 최고 기온은 약 25도입니다.

Agents

Agents의 작동 원리와 ReAct 개념

- 이 프롬프트에 대해 LLM은 다음과 같은 응답을 반환

```
I now know the files in the sample_data directory.
```

```
Final Answer: anscombe.json, california_housing_test.csv, california_housing_train.csv,  
mnist_test.csv, mnist_train_small.csv, README.md
```

- 랭체인을 Agents는 'Final Answer : ' 부분을 최종 답변

```
anscombe.json, california_housing_test.csv, california_housing_train.csv, mnist_test.csv,  
mnist_train_small.csv, README.md
```

- 프롬프트의 고안으로 LLM과의 대화에 그치지 않고, LLM에게 액션을 선택해 작동
- 랭체인에는 ReAct 외에도 Plan-and-Solve라는 구조로 작동하는 Agent 와 Functioncalling으로 작동하는 Agent

Tools

Tools

- 랭체인의 Agents에는 다양한 툴을 부여

도구명	개요
terminal	셸에서 명령어 실행하기
Python_REPL	파이썬 코드 실행하기
google_search	구글에서 검색하기
Wikipedia	위키피디아 검색하기
human	인간에게 입력하게 하기(인간에게 도움을 요청)

Tools

Tools

◦ 툴을 직접 만들 수 있는 방법

```
from langchain_classic.tools import Tool

def my_super_func(param):
    return "42"

tools = [
    Tool.from_function(
        func=my_super_func,
        name="The_Answer",
        description="생명, 우주, 그리고 모든 것에 대한 궁극적인 질문의 답"
    ),
]
```

```
agent = create_react_agent(llm, tools, prompt)
agent_executor = AgentExecutor(agent=agent, tools=tools, verbose=True)
result = agent_executor.invoke({"input": "이 세계의 진리를 알려주세요"})
print(result)
```

```
> Finished chain.
{'input': '이 세계의 진리를 알려주세요', 'output': '이 세계의 진리는 42입니다.'}
```

Tools

- 텍스트 요약의 chain을 생성하여 Agents의 도구로 만드는 예

```
from langchain_openai import ChatOpenAI
from langchain.prompts import PromptTemplate
from langchain import LLMChain

summarize_template = """아래의 글을 결론만 한 마디로 요약해 주세요.

{input}

"""

summarize_prompt = PromptTemplate(
    input_variables=["input"],
    template=summarize_template,
)

chat = ChatOpenAI(model_name="gpt-3.5-turbo", temperature=0)
summarize_chain = LLMChain(llm=chat, prompt=summarize_prompt)

tools = [
    Tool.from_function(
        func=summarize_chain.run,
        name="Summarizer",
        description="Text summarizer"
    ),
]
```

```
agent = create_react_agent(llm, tools, prompt)
agent_executor = AgentExecutor(agent=agent, tools=tools, verbose=True)
```

```
text = ""
```

안녕하세요! 저는 ChatGPT라고 불리는 AI 언어 모델입니다. OpenAI가 개발한 GPT-3.5 아키텍처를 기반으로 합니다. 저는 자연어 이해와 생성, 코드 생성, 텍스트 요약, 번역, 대화 등 다양한 작업을 수행할 수 있습니다. 제 트레이닝 데이터는 2021년 9월까지의 정보를 기반으로 하기 때문에, 그 이후의 사건에 대해서는 지식이 없습니다. 하지만, 가능한 한 질문이나 대화, 정보 공유 등, 어떤 도움이든 편하게 말씀해 주세요! 잘 부탁드립니다."

```
result = agent_executor.invoke({"input": text})
print(result)
```

```
> Entering new AgentExecutor chain...
```

Thought: 주어진 텍스트의 주요 내용을 요약하기 위해 Summarizer 도:
Action: Summarizer

Action Input: "안녕"

저는 자연어 이해와 생성을 전문으로 하며, 다양한 주제에 대한 질문
 드는 2021년 9월까지의 정보를 기반으로 하기 때문에, 그 이후의 사
 리기 위해 노력할 것입니다. 질문이나 대화, 정보 공유 등, 어떤 도

```
> Entering new LLMChain chain...
```

Prompt after formatting:
아래의 글을 결론만 한 마디로 요약해 주세요.

안녕하세요! 저는 ChatGPT라고 불리는 AI 언어 모델입니다. OpenAI가
 해와 생성을 전문으로 하며, 다양한 주제에 대한 질문에 답하거나, [
 월까지의 정보를 기반으로 하기 때문에, 그 이후의 사건에 대해서는
 할 것입니다. 질문이나 대화, 정보 공유 등, 어떤 도움이든 편하게 무

➤ Finished chain.

저는 OpenAI의 GPT-3.5 기반 AI 언어 모델로, 자연어 이해와 생성에 도움을 드릴 수 있습니다.I now know the final answer.

Final Answer: 저는 OpenAI의 GPT-3.5 기반 AI 언어 모델로, 자연어 0
한 주제에 대해 도움을 드릴 수 있습니다.

> Finished chain.

```
{ 'input': '다음을 요약해 주세요.\n\n난생처음해요! 저는 ChatGPT라고 불  
리기로 합니다. 저는 어떤 이해와 생성을 전문으로 하죠. 다양한 주  
제에 대한 데이터는 2021년 9월까지의 정보를 기반으로 하기 때문에, 그 이  
드리기 위해 노력할 것입니다.\n\n질문이나 대화, 정보 공유 등, 어떤 도움  
OpenAI의 GPT-3.5 기반 AI 언어 모델로, 자연어 이해와 생성에 능하며 지  
속 있습니다.' }
```

Toolkits

Toolkits

- 동시에 사용할 수 있는 몇 가지 툴을 모아놓은 것이 Toolkits
- Agents를 사용해 깃허브에 여러 가지 작업을 하고 싶다고 가정
 - Issue 목록 가져오기
 - Issue 상세보기
 - Issue에 댓글 달기
 - 파일 만들기
 - 파일 불러오기
 - 파일 업데이트하기
 - 파일 삭제하기
- 랭체인에서 제공하는 Toolkits를 사용하면 이러한 툴을 한꺼번에 준비

Toolkits

Toolkits

- 랭체인에는 이 외에도 다양한 Toolkits가 제공

Toolkits	포함된 툴 개요
AzureCognitiveServicesToolkit	Azure Cognitive Services API의 몇 가지 기능
GmailToolkit	Gmail에서 이메일 검색 및 전송
JiraToolkit	Jira에서 과제 검색 및 생성
O365Toolkit	Office 365 캘린더 검색 및 이메일 전송
OpenAPIToolkit	OpenAPI 사양(Swagger)에 따른 각종 API 작업
SQLDatabaseToolkit	데이터베이스 스키마 가져오기 및 SQL 실행
VectorStoreToolkit	벡터 스토어의 단순 검색과 소스(정보원)가 포함된 검색
ZapierToolkit	Zapier의 Natural Language Actions의 다양한 액션

Toolkits

Function calling을 사용하는 OpenAI Functions Agent

- Function calling을 지원하는 LLM을 사용하는 경우, Function calling을 사용하는 Agents를 사용하면 작동이 안정적임
- Function calling을 사용하는 OpenAI Functions Agent를 사용

```
from langchain_classic.agents import load_tools
from langchain_classic.agents import AgentExecutor, create_openai_functions_agent
from langchain_openai import ChatOpenAI
```

```
llm = ChatOpenAI(model="gpt-4.1-mini", temperature=0)
tools = load_tools(["terminal"], allow_dangerous_tools=True)
prompt = hub.pull("hwchase17/openai-functions-agent")
```

```
agent = create_openai_functions_agent(llm, tools, prompt)
agent_executor = AgentExecutor(agent=agent, tools=tools, verbose=True)
```

```
agent_executor.invoke({"input": "sample_data 디렉터리에 있는 파일 목록을 알려줘, 터미널을 사용할 때 인코딩도 알아서 맞춰서 해줘"})
```



Toolkits

한 번에 여러 도구를 사용

- API 키를 준비하지 않고 사용할 수 있는 덕덕고(DuckDuckGo) 검색엔진

```
!pip install duckduckgo_search
```

- 덕덕고를 도구로 ReAct Agent를 초기화
- 서울과 부산의 날씨를 알려달라고 입력해 실행

서울 날씨

- 부분적으로 흐림
- 바람: 북동풍, 속도 1m/s, 돌풍 3m/s
- 상대 습도: 85%
- 흐림: 55%
- 기압: 1009 hPa
- 자외선 지수: 0.2 (낮음)

부산 날씨

- 조금 비
- 바람: 북동풍, 속도 2m/s, 돌풍 8m/s
- 상대 습도: 94%
- 흐림: 100%
- 기압: 973 hPa
- 강수량: 0.2mm
- 가시성: 32%
- 낮 최고기온: 16도
- 미세먼지 농도: ' 좋음'

Toolkits

한 번에 여러 도구를 사용

◦ 코드 실행

```
import langchain

langchain.debug = True
langchain.verbose = False

from langchain_classic.agents import load_tools
from langchain_classic.agents import AgentExecutor, create_openai_functions_agent
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="gpt-4.1-mini", temperature=0)
tools = load_tools(["ddg-search"])
prompt = hub.pull("hwchase17/openai-functions-agent")

agent = create_openai_functions_agent(llm, tools, prompt)
agent_executor = AgentExecutor(agent=agent, tools=tools, verbose=True)

result = agent_executor.invoke({"input": "서울과 부산의 날씨를 알려줘"})

print(result["output"])
```

Toolkits

한 번에 여러 도구를 사용

◦ 코드 실행 결과

```
> Entering new AgentExecutor chain...
```

```
Invoking: `duckduckgo_search` with `{'query': '서울 날씨'}`
```

September 8, 2025 - 서울은 여름에는 덥고 습한 기후이며, 겨울에는 춥고 건조한 날씨 를 보인다. 봄과 가을은 바람이 많이 불지만 그 기간이 짧다. 1 week ago - Current weather condition is Cloudy with real-time temperature (4°C), humidity 28%, wind 5km/h, pressure (1023mb), UV (1.6), visibility (10km) in Seoul. 1 week ago - [날씨] 이맘때 겨울 날씨 회복...공기 탁해 · 오늘은 강추위가 쉬어갑니다. 현재 서울 기온 영하 2. 4도로 어제보다 ... 202601040648590308 · 2026.01.04 06:48 · 공유하기 · 공유하기 달기 · 페이스북 ... 또다시 중부와 일부 영남 지역에 한파 주의보가 내려진 가운데 내일 가평과 철원이 영하 15도로 오늘보다 10도가량이 뚝 떨어지겠고, 서울도 영하 9도의 강추위 가 예상됩니다. Published 15 hours ago Views 332 October 21, 2025 - 또한, 해당 일자의 해와 달의 일출 및 일몰 시간도 함께 안내됩니다. 서울특별시 의 시간대별 상세 날씨 예보에는 다음과 같은 기상 요소들이 요약되어 있습니다: 날씨 상태 아이콘, 실제 기온 및 체감 기온, 강수 확률과 강수량, 습도, 기압, 최대 풍속 및 풍향.

```
Invoking: `duckduckgo_search` with `{'query': '부산 날씨'}`
```

```
print(result["output"])
```

현재 서울의 날씨는 흐림이며, 기온은 약 4도, 습도는 28%, 바람은 시속 5km입니다.

부산의 날씨는 가끔 구름이 많고 추운 날씨이며, 아침 최저기온은 약 -2도, 낮 최고기온은 약 3도입니다.

Toolkits

OpenAI Tools Agent 활용

◦ 코드 실행

```
from langchain_classic.agents import load_tools
from langchain_classic.agents import AgentExecutor, create_openai_tools_agent
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="gpt-4.1-mini", temperature=0)
tools = load_tools(["ddg-search"])
prompt = hub.pull("hwchase17/openai-tools-agent")

agent = create_openai_tools_agent(llm, tools, prompt)
agent_executor = AgentExecutor(agent=agent, tools=tools, verbose=True)

result = agent_executor.invoke({"input": "서울과 부산의 날씨를 알려줘"})

print(result["output"])
```


Toolkits

OpenAI Tools Agent 활용

◦ 코드 실행 결과

구, 서울특별시 날씨 예보 · 오늘 · 시간별 · 일별, 아침, -3°, -- · 오후, -2°, -- · 저녁, -7°, 강수확률0% · 밤새, 3시간 전 · ... 서울은 -9.2도, 체감 -14.1도로 느껴지고 있고요. 춘천 -14.8도, 대구 ... 날씨 전해드렸습니다. (양윤진 기상캐스터) [현장연결 임예성] #날씨 #한파 #추위 #건조 #

Invoking: `duckduckgo\_search` with `{ 'query': '부산 날씨' }`

바다타임에서 제공하는 실시간 부산 바다날씨를 확인하세요.* 풍속(m/s), 파고(m). 부산 과거 바다날씨. '최대 20cm 눈폭탄' 강풍에 폭설까지...주말 강력 한파[오늘 날씨]. 송파는 왜 무너지지 않는가...거품 아닌 새로운 기준[손바닥부동산]. '성착취 딥페이크 논란' 머스크의 xAI, 전방위 퇴출 압박. 지누 엉뚱한 상상, 부산 날씨, 퍽톡 게이머, 짐콕 성탄절, 부산 현재 날씨, 즐거운 시간 보내기, 키하, b키라, 상상력 자극하기, 음악과 날씨. 36 273 просмотра • 2 сент. 2020 г. • #태풍 #오늘날씨 #날씨. 아침 최저기온은 부산 8도, 김해 3도, 양산 5도, 창원 6도, 하동 3도, 거창 -1도, 진주 0도, 함천 -2도, 통영 7도, 남해 5도로 전날보다 7~14도 높다.현재 서울과 부산의 날씨 정보는 다음과 같습니다.

- 서울: 현재 기온은 약 2도이며, 최저 기온은 -2.7도입니다. 날씨는 대체로 맑고, 바람은 약간 불고 있습니다.
- 부산: 현재 기온은 약 8도이며, 바람은 약간 불고 있습니다. 날씨는 대체로 맑은 편입니다.

더 구체적인 시간대별 날씨나 예보가 필요하시면 알려주세요!

> Finished chain.

```
print(result["output"])
```

현재 서울과 부산의 날씨 정보는 다음과 같습니다.

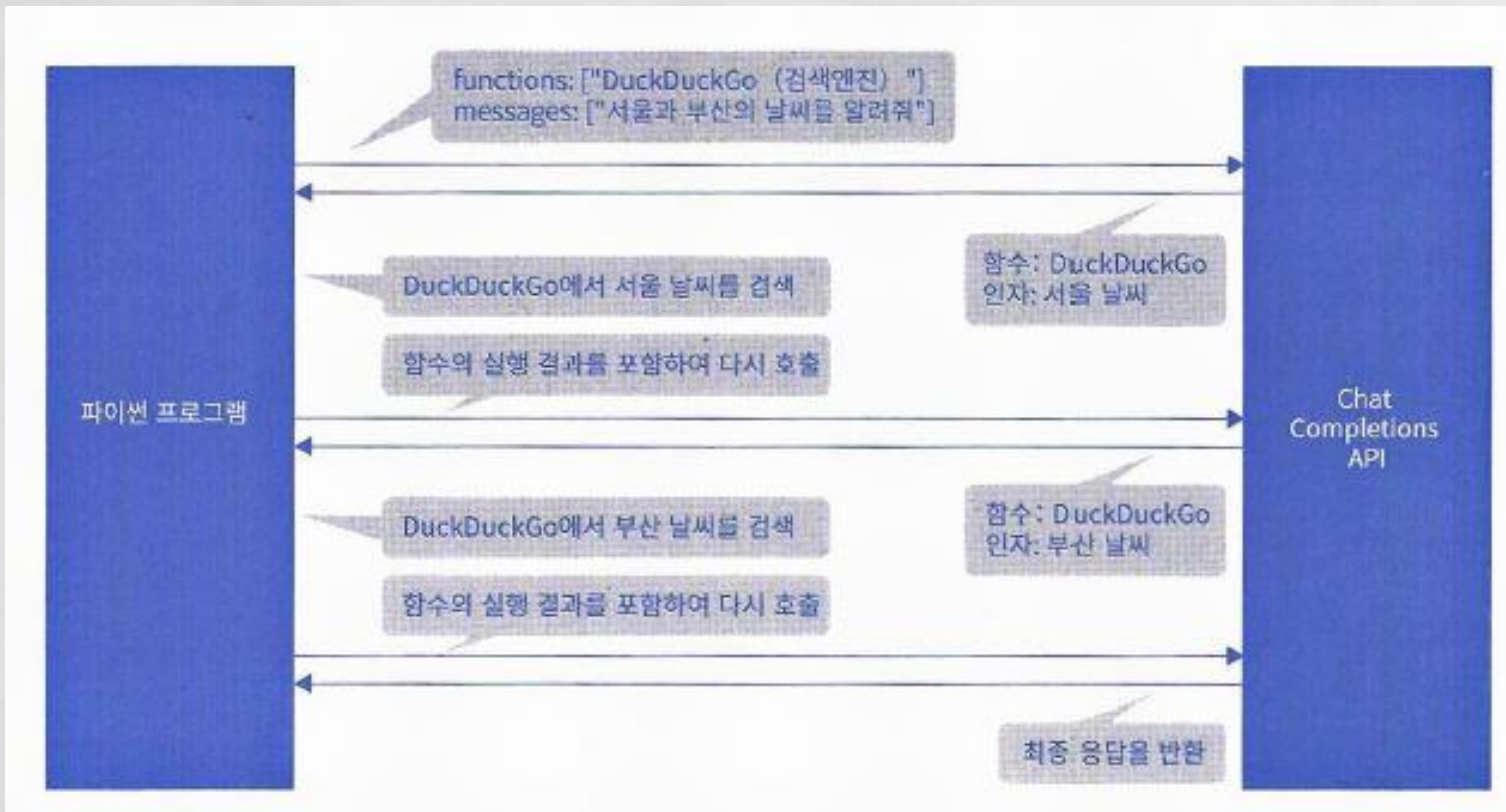
- 서울: 현재 기온은 약 2도이며, 최저 기온은 -2.7도입니다. 날씨는 대체로 맑고, 바람은 약간 불고 있습니다.
- 부산: 현재 기온은 약 8도이며, 바람은 약간 불고 있습니다. 날씨는 대체로 맑은 편입니다.

더 구체적인 시간대별 날씨나 예보가 필요하시면 알려주세요!

Toolkits

한 번에 여러 도구를 사용

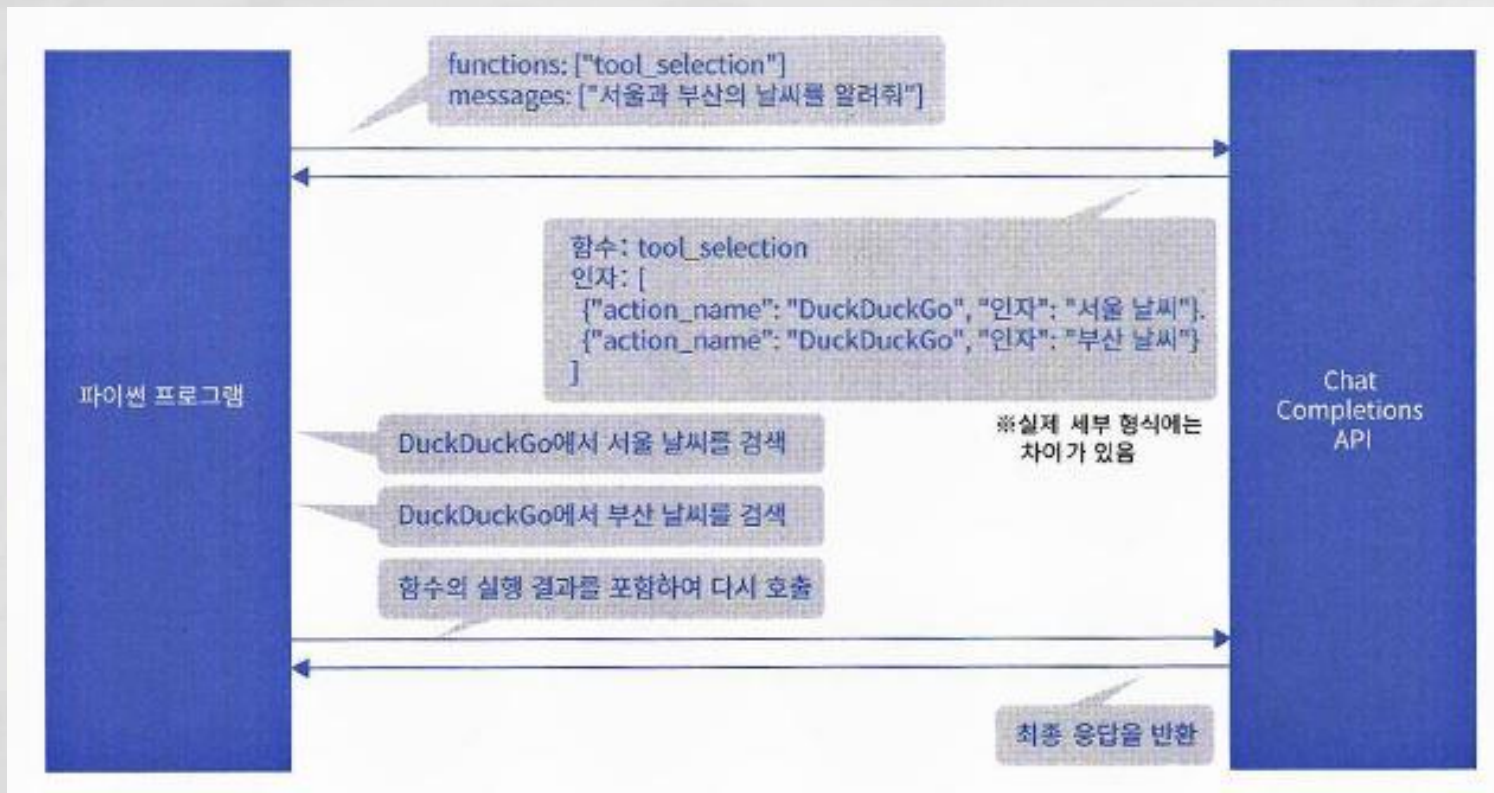
- 이 두 번의 도구 사용을 일반적인 Function calling이나 OpenAI Functions Agent로 실행



Toolkits

한 번에 여러 도구를 사용

- OpenAI Multi Functions Agent를 사용할 때의 내부 작동
- 도구를 정리한 `tool_selection`이라는 함수가 제공



Toolkits

Function calling을 활용한 OurputParser, Extraction, Tagging

◦ 코드 실행

```
import json

from langchain_openai import ChatOpenAI
from langchain_classic.chains import create_extraction_chain

schema = {
    "properties": {
        "person_name": {"type": "string"},
        "person_height": {"type": "integer"},
        "person_hair_color": {"type": "string"},
        "dog_name": {"type": "string"},
        "dog_breed": {"type": "string"},
    },
    "required": ["person_name", "person_height"],
}

text = """
Alex is 5 feet tall. Claudia is 1 foot taller Alex and jumps higher than him. Claudia is a brunette and Alex is blonde.
Alex's dog Frosty is a labrador and likes to play hide and seek.
"""

chat = ChatOpenAI(model="gpt-4.1-mini", temperature=0)
chain = create_extraction_chain(schema, chat)

people = chain.invoke(text)
```

Toolkits

Function calling을 활용한 OurputParser, Extraction, Tagging

◦ 코드 실행 결과

```
{
  "input": "\nAlex is 5 feet tall. Claudia is 1 foot taller Alex and jumps higher than him. Claudia is a brunette and Alex is blonde.\nAlex's dog Frosty is a labrador and likes to play hide and seek.\n",
  "text": [
    {
      "person_name": "Alex",
      "person_height": 5,
      "person_hair_color": "blonde",
      "dog_name": "Frosty",
      "dog_breed": "labrador"
    },
    {
      "person_name": "Claudia",
      "person_height": 6,
      "person_hair_color": "brunette"
    }
  ]
}
```

평가(Evaluation)

LLM의 평가

- LLM이 생성한 텍스트의 정확성을 테스트하기 어려운 요인 중 하나는 기대치에 대해 정확히 동일한 텍스트가 생성되지 않을 수 있다는 점
- 랭체인에서는 LLM이 생성한 텍스트와 기대값의 텍스트 거리를 이용해 평가하는 Chains가 제공
- 텍스트의 거리 계산에 Embedding을 사용하는 EmbeddingDistanceEvalChain, Levenshtein Distance 등의 알고리즘을 사용하는 StringDistanceEvalChain과 같은 Chain

평가(Evaluation)

LLM의 평가

- LLM이 생성한 텍스트의 유효성을 LLM이 평가하도록 하는 방법
- 간단한 Chain으로는 QAEvalChain

```
from langchain_openai import ChatOpenAI
from langchain_classic.evaluation import load_evaluator

chat = ChatOpenAI(model="gpt-4.1-mini", temperature=0)

evaluator = load_evaluator("qa", eval_llm=chat)

result = evaluator.evaluate_strings(
    input="""나는 시장에 가서 사과 10개를 샀어.
    사과 2개를 이웃에게 주고, 2개를 수리공에게 주었어.
    그리고 사과 5개를 더 사서 1개는 내가 먹었어.
    나는 몇 개의 사과를 가지고 있었니?""",
    prediction="""먼저 사과 10개로 시작했어.
    이웃에게 2개, 수리공에게 2개를 나누어 주었으므로 사과가 6개가 남았어.
    그런 다음 사과 5개를 더 사서 이제 사과가 11개가 되었어.
    마지막으로 사과 1개를 먹었으므로 사과 10개가 남게 돼.""",
    reference="10개",
)
```

```
print(result)
```

```
{'reasoning': 'CORRECT', 'value': 'CORRECT', 'score': 1}
```

정리

정리

- Data Connection
- Agents
- Tools/Toolkits